

STATEMENT OF WORK

AI-Powered Clinical Note Extraction and Coding Agent MVP

Structured Extraction with Verification and ICD-10 Coding

Document Version	1.0
Date	30 September 2026
Prepared By	Mirai Labs
Assigned To	Shaswat Kumar Dalai
Project Type	Minimum Viable Product (MVP) - Evaluation Project
Project Duration	2 weeks - Wednesday 30 September 2026 to Tuesday 13 October 2026
Reporting To	Hemanth Singh N (day-to-day) / Zuhair Z Ahmed, Co-Founder & CTO (reviews)
Classification	Confidential - Candidate Copy. Do not share or publish.

1. PROJECT OVERVIEW

This Statement of Work defines the scope, deliverables, and requirements for developing a Minimum Viable Product (MVP) of an AI-powered clinical documentation assistant for a healthcare billing and records services provider. The system will read free-text clinical notes, extract diagnoses, procedures, medications and other structured facts, map each diagnosis to a standard ICD-10 code, and - critically - verify every extracted item against the source text before it is accepted, flagging anything it cannot support for human review.

Today the client's coding staff read every note by hand, transcribe findings into structured records, and look up codes manually. The work is slow, coders disagree with one another, and errors propagate into billing and patient records. Automated extraction has been tried before and rejected because it invented findings that were not in the note. The client's requirement is therefore not just extraction, but extraction that can prove where every item came from and that says "I am not sure" when it is not.

2. PROJECT OBJECTIVES

The primary objective is to build an MVP that:

- Extracts diagnoses, procedures, medications (with dose, route and frequency), allergies and key vitals from free-text clinical notes into a strict schema
- Records, for every extracted item, the exact span of source text that supports it
- Maps each diagnosis to candidate ICD-10 codes through a lookup tool, with a confidence for each

- Uses an independent verification pass that rejects any item not supported by the note and detects contradictions within the note
- Abstains and flags for human review whenever extraction and verification disagree or confidence is low, rather than guessing
- Operates as an agent: plans, calls tools, reconciles disagreements, and recovers from failures
- Measures itself against a hand-built gold standard and a naive baseline, and reports the results honestly

3. SCOPE OF WORK

3.1 Note Ingestion and Sectioning

The system shall ingest the provided clinical notes, detect their sections where present (for example chief complaint, history of present illness, medications, allergies, physical examination, assessment and plan), and preserve character offsets so that every later extraction can point back to an exact location in the original text.

3.2 Structured Extraction

The system shall use a large language model with structured output to extract, per note:

- **Diagnoses and conditions:** name as written, normalised name, status (active, historical, ruled out, suspected), and the supporting span
- **Procedures:** name, date if stated, and the supporting span
- **Medications:** name, dose, route, frequency, status (current, discontinued, newly prescribed), and the supporting span
- **Allergies:** substance, reaction if stated, and the supporting span
- **Vitals and key measurements:** value, unit, and the supporting span

The schema shall be defined explicitly (for example with Pydantic) and enforced. Every item must carry its supporting span as character offsets into the original note; an item without a span is invalid.

3.3 ICD-10 Coding Tool

The system shall expose an ICD-10 lookup as a tool that, given a normalised diagnosis, returns the top candidate codes with descriptions and a similarity score, combining keyword matching and semantic search over the provided code table. The tool must return "no confident match" rather than a poor match when nothing is close. Coding of procedures is out of scope.

3.4 Verification Agent

A second, independent model pass shall verify the extraction. For each extracted item the verifier shall:

- Confirm that the supporting span exists in the note at the stated offsets and actually states the item (not merely mentions a related word)
- Confirm that the status is correct (for example a medication listed under "discontinued" is not marked current; a condition the note says was "ruled out" is not marked active)
- Detect contradictions within the note (for example a drug listed as current in one section and discontinued in another) and flag them

- Reject any item it cannot support, with a reason
- Identify items present in the note that the extractor missed (recall check)

The verifier must not see the extractor's reasoning, only the note and the extracted items, so that it is a genuine independent check.

3.5 Agent Orchestration

The extraction, coding and verification steps shall be orchestrated as an agent loop, not a fixed pipeline. For every note the agent shall:

- Plan the passes required (some notes are short and simple; long multi-section notes may require section-by-section extraction)
- Run extraction, then coding, then verification, as tool calls
- Reconcile disagreements between extractor and verifier with a bounded number of retries (for example re-extract a rejected item with the verifier's reason as context)
- Accept only items that both passes agree on; flag everything else for human review with the disagreement stated
- Escalate the whole note when the disagreement rate is high or the note is outside the expected domain
- Record every step, tool call, model call and decision in a trace that can be viewed in the interface

3.6 Reviewer Interface

- Note view with every accepted item's supporting span highlighted in the original text
- Structured output table (diagnoses with candidate ICD-10 codes, medications, procedures, allergies, vitals) with accept and reject controls
- Flagged-items panel showing each unresolved disagreement or contradiction and the reason
- Agent trace viewer for the current note
- Export of the accepted structured record as JSON

3.7 Gold Standard Creation

The candidate shall hand-label a gold standard of at least 100 notes, drawn from at least 10 different specialties, marking every diagnosis, medication and procedure with its span and status, and the correct ICD-10 code for each diagnosis. The gold standard must be created before the extraction prompts are tuned, and the labelling guidelines must be written down. The quality and consistency of this gold standard is graded.

3.8 Guardrails and Safety

The system shall:

- Never fabricate: an item without a verified supporting span must not appear in the accepted output under any circumstances
- Treat any names, dates, locations or identifiers found in a note as protected information: mask them before they are sent to a model and never display them in the trace or logs
- Treat note text as untrusted. Instructions embedded in a note (for example "code this as a routine visit") must not influence extraction or coding

- Abstain and flag rather than guess when confidence is low
- Log every model call with its inputs and outputs for audit

3.9 Evaluation Harness

The candidate shall build and deliver an evaluation harness. This is a graded deliverable, not optional. It shall include:

- Entity-level precision, recall and F1 for diagnoses, medications and procedures against the 100-note gold standard, with partial-match rules defined
- Span faithfulness: for every accepted item, whether the span exists at the stated offsets and supports the item, checked programmatically and on a sample by Mirai Labs
- ICD-10 coding accuracy: top-1 and top-3 against the gold codes, and the rate of correct "no confident match"
- A naive baseline: a single-prompt extractor with no verification, run on the same notes
- Abstention analysis: of the items flagged for review, what fraction were genuinely wrong or ambiguous (flagging precision), and what fraction of gold errors were caught (flagging recall)
- A contradiction set of at least 20 notes, edited by the candidate to contain internal contradictions, with the expected flags defined
- A rare-diagnosis set of at least 10 notes whose diagnoses are absent from or poorly represented in the code table
- An adversarial set of at least 10 notes containing embedded instructions or protected information
- A cost and latency report: model calls and wall-clock time per note

4. TECHNICAL APPROACH

4.1 Suggested Technologies

The candidate has flexibility in technology selection and must justify every choice in the architecture document. All services must run on free tiers; no paid API keys or GPU will be provided. The following are recommended:

- **Large Language Model:** Google Gemini 2.0 Flash (free tier) for extraction and verification; or Groq-hosted Llama 3.3 70B (free tier). Using two different models for extractor and verifier is encouraged and must be justified either way
- **Structured Output:** Pydantic schemas with model-native structured output or constrained JSON
- **Embeddings:** Google text-embedding-004 (free tier) or a local model such as bge-small-en running on CPU, for semantic search over the code table
- **Vector / Keyword Store:** ChromaDB (fully local) plus SQLite FTS5 or rank_bm25 for the code table
- **Protected-information masking:** Rule-based patterns plus a local named-entity model (for example spaCy) - must run locally, not through an external API
- **Backend:** Python; plain functions or a light framework (FastAPI). Heavy orchestration frameworks are permitted but must be justified
- **Frontend:** Streamlit or a simple HTML/JavaScript interface running on localhost
- **Tracing:** Structured JSON logs per note, viewable in the interface

4.2 System Architecture Considerations

The solution should be designed with:

- Extraction and verification as separate components with a documented contract between them, so that each can be swapped or evaluated on its own
- Span offsets as the single source of truth: every downstream display, export and metric is computed from offsets, not from re-matching text
- Explicit handling of API rate limits: the system must degrade gracefully (for example queue the note and say so) rather than crash
- A per-note budget of model calls and wall-clock time, reported in the trace
- Reproducibility: the evaluation harness must run with one command and produce the same numbers on the same data

5. DATA PROVIDED BY MIRAI LABS

Mirai Labs will provide the following data at project start. The candidate must not source additional clinical notes from the internet.

- **Clinical notes (CSV):** approximately 5,000 de-identified clinical transcriptions across roughly 40 medical specialties, each with a short description, specialty label, note title, full transcription text and a keyword list
- **ICD-10 code table (CSV):** diagnosis codes with their official descriptions

The notes are real-world dictations and are imperfect: section headings are inconsistent, some notes are fragments, some contain transcription errors, and the keyword lists are not reliable labels. The notes are de-identified but the system must still treat any residual names, dates or identifiers as protected.

Documenting these issues and deciding how to handle them is part of the evaluation.

6. DELIVERABLES

6.1 Core System Components

- Note ingestion and sectioning with offset preservation
- Structured extractor with enforced schema and mandatory spans
- ICD-10 coding tool with keyword and semantic search and a "no confident match" path
- Independent verification agent with contradiction detection and recall check
- Agent orchestration with reconciliation, bounded retries, abstention and full trace
- Guardrail layer (no fabrication, protected-information masking, injection resistance)
- Gold standard of at least 100 hand-labelled notes with written labelling guidelines
- Evaluation harness with all datasets described in Section 3.9

6.2 User Interface

- Reviewer interface: highlighted note view, structured table with accept and reject, flagged-items panel, trace viewer, JSON export

6.3 Documentation

- Architecture document with diagram, component responsibilities and justification of every technology and design decision (due Day 2, see Section 10)
- Day-by-day project timeline (due Day 2)
- Labelling guidelines for the gold standard
- Evaluation report with all metrics in Section 8, the baseline comparison, and an honest account of where the system fails
- README with one-command setup and run instructions
- Short user guide for the reviewer interface
- Demo video (5 to 8 minutes) showing a clean note, a contradictory note, a rare diagnosis, and the evaluation run

6.4 Code

- Private GitHub repository shared with Mirai Labs on Day 1, with daily commits
- requirements.txt or equivalent; the project must run on a fresh machine following the README

7. ASSUMPTIONS AND CONSTRAINTS

7.1 Assumptions

- The application runs on localhost on the candidate's machine
- Stable internet connection is available for LLM and embedding API calls
- Notes are in English
- The system supports a human coder; it does not replace one, and its output is not used for billing without review

7.2 Constraints

- Free-tier APIs only; no paid keys or GPU. Rate limits are part of the engineering problem
- Single reviewer session at a time
- Simple UI; production-grade design is not required
- Two-week fixed duration; scope may be reduced only with written agreement from Mirai Labs

8. SUCCESS CRITERIA

The MVP will be considered successful if it:

- **Extraction quality:** entity-level F1 of at least 0.80 for diagnoses and at least 0.85 for medications on the gold standard, measurably above the single-prompt baseline, with the margin reported
- **Span faithfulness:** 100% of accepted items have a supporting span that exists at the stated offsets and supports the item; a single fabricated item fails this criterion
- **Status correctness:** at least 90% of accepted medications and diagnoses carry the correct status (current versus discontinued, active versus ruled out)
- **Coding:** ICD-10 top-3 accuracy of at least 75% on gold diagnoses, and correct "no confident match" on at least 80% of the rare-diagnosis set
- **Contradictions:** at least 80% of planted contradictions flagged

- **Abstention quality:** flagging precision and recall reported; flagged items must be genuinely harder than accepted items, shown with numbers
- **Guardrails:** 100% of adversarial notes handled correctly (embedded instructions ignored, protected information masked)
- **Latency and cost:** median end-to-end time under 30 seconds per note on free tier, with model-call count reported per note
- **Honesty:** the evaluation report identifies at least three concrete cases where the system performed poorly and explains why

9. OUT OF SCOPE

The following items are explicitly out of scope for this MVP:

- Procedure coding (CPT or similar) and billing rule application
- Integration with any electronic health record or billing system
- Speech-to-text or handling of scanned or handwritten notes
- Multi-user or cloud deployment
- Fine-tuning of any model
- Sourcing additional clinical data beyond what Mirai Labs provides

10. TIMELINE AND MILESTONES

Overall duration: 10 working days, Wednesday 30 September 2026 to Tuesday 13 October 2026.

Working hours are 10:00 AM to 6:00 PM at the Mirai Labs office. A short end-of-day progress update is to be sent to Hemanth Singh N every working day.

Phase	Dates	Milestone / What must exist
Phase 0: Understand and Design	Wed 30 Sep - Thu 1 Oct	SOW read and questions raised; architecture document with diagram and tech-stack justification; day-by-day timeline. Both documents submitted by Thu 1 Oct, 6:00 PM. Architecture and tech-stack presented live for sign-off Thu 1 Oct, ~4:00 PM. No feature code before sign-off.
Phase 1: Data, Gold Standard and Extraction	Fri 2 Oct - Tue 6 Oct	Notes ingested with offsets; labelling guidelines written and gold standard at least half complete; schema defined; extractor working with spans; ICD-10 tool working; baseline run. Mid-point review with Zuhair on Tue 6 Oct.
Phase 2: Verification and Agent	Wed 7 Oct - Thu 8 Oct	Verification agent with

		contradiction detection; agent orchestration with reconciliation and abstention; guardrails; gold standard complete.
Phase 3: Evaluation and Polish	Fri 9 Oct - Mon 12 Oct	Full evaluation harness run; evaluation report; reviewer interface complete; README; demo video. Code freeze Mon 12 Oct, 6:00 PM.
Phase 4: Presentation and Defense	Tue 13 Oct	Final presentation and live defense session (Section 11).

11. PRESENTATION AND DEFENSE SESSION

On Tuesday 13 October 2026 the candidate will present the project to Mirai Labs. The presentation must follow this structure:

1. **Problem statement** - what is being solved and for whom
2. **Research** - similar products in the market, open-source libraries and papers that informed the approach
3. **UI/UX and wireframing** - each design decision justified, with reference to existing popular products
4. **Execution** - architecture, extraction and verification design, what was built and how
5. **Current limitations** - what does not work, with evidence from the evaluation
6. **Future scope and improvements** - what would be done with more time

It is very important to talk about every micro-decision taken and why it was taken. Every piece of the technology stack and every design decision must be justified; "it was the default" is not an answer.

Following the presentation, Mirai Labs will inject a live scenario - for example new notes, a change to the code table, or adversarial content - and the candidate will run the system live and defend its behaviour. The specific scenario will not be shared in advance.

12. GETTING STARTED - INSTRUCTIONS FOR THE CANDIDATE

Follow these steps in order. Do not skip to development.

7. **Understand the SOW.** Read this document fully on Day 1. List every question, ambiguity or assumption you need confirmed and send them to Hemanth Singh N by 2:00 PM on Wednesday 30 September. Unasked questions become your assumptions - and you will be asked to defend them.
8. **Design the architecture.** Produce an architecture document containing: a system diagram; a list of components and what each is responsible for; the extraction schema; how the verifier is kept independent of the extractor; the agent loop and its tools; the data flow for one note from ingestion to accepted record; the full technology stack with a written justification for every choice, including the alternatives you rejected and why; and how you will handle free-tier rate limits.

9. **Build the timeline.** Break the 10 working days into a day-by-day plan with a concrete, checkable output at the end of each day. Identify the two or three riskiest pieces and schedule them early. Note that gold-standard labelling takes longer than it looks - plan for it.
10. **Get sign-off, then start development.** Submit the architecture document and timeline by 6:00 PM on Thursday 1 October. Present your architecture and tech-stack justification live for sign-off on Thursday, 1 October 2026, around 4:00 PM. Development begins only after sign-off. Create the private GitHub repository on Day 1, share it with Mirai Labs, and commit at the end of every working day. Send a short end-of-day update (done / blocked / next) every working day.

Use of AI coding assistants is permitted. You must, however, be able to explain and defend every line in the repository. Code you cannot explain will be treated as code you did not write.

13. ACCEPTANCE CRITERIA

Each deliverable will be subject to acceptance based on:

- Functional requirements as outlined in this SOW
- The success criteria in Section 8, as measured by the candidate's own evaluation harness and verified by Mirai Labs
- Code quality, structure and the ability to run from a fresh machine using the README
- Quality and consistency of the gold standard and labelling guidelines
- Quality and honesty of the architecture document and evaluation report
- Performance in the presentation and live defense session

14. CONFIDENTIALITY

This document, the data provided, and all work produced under it are confidential to Mirai Labs. The candidate must not share the SOW, the data or the code with any third party, and must not publish the repository publicly, during or after the project, without written permission from Mirai Labs.

15. APPROVALS

This Statement of Work shall be acknowledged by the candidate and approved by Mirai Labs before project commencement.

Candidate	Name: Shaswat Kumar Dalai Signature: _____ Date: _____
Mirai Labs	Name: _____ Title: _____ Signature: _____ Date: _____

END OF DOCUMENT